

All Projects

Real-Time Interview Questions & Answers

Prepared from your actual project experience: React Native, Expo, Spring Boot, Node.js, MySQL, Docker, AWS EC2, OTP, Payments, Admin Dashboards, and Unity/Game projects.

Interview Profile Summary

Main Profile	Full-stack / Mobile App Developer
Frontend	React Native, Expo, React JS / Expo Web
Backend	Spring Boot, Node.js Express
Database	MySQL
Deployment	Docker Compose, AWS EC2, Render, GitHub Actions
Integrations	Twilio OTP, Razorpay Payments, JWT, Role-Based Access, Notifications

How to Use This PDF

Use this as your interview speaking notes. Do not memorize every word. Understand the meaning and answer naturally using examples from your own projects.

- For fresher or junior interviews, focus on React Native, API calls, MySQL, authentication, and project explanation.
- For full-stack interviews, focus on Spring Boot/Node, JWT, role-based access, payment verification, Docker, AWS, and debugging.
- For real-time project discussion, explain problems you solved: CORS, 500 errors, port conflicts, OTP failure, APK build issues, and deployment issues.

Project Experience You Can Mention

- **All Community Matrimony:** Bride/groom profiles, vendor services, admin approval, subscriptions, payment flow, Telugu/English language support.
- **TraceMyBus:** School bus tracking with admin, driver, parent, live location details, pickup/drop status, SOS, and notifications.
- **Nari Samman:** E-commerce app for honey, spices, sarees, handicrafts, COD, product listing, and admin management.
- **Gated Community / All Services:** Resident/service booking apps with OTP login, dashboards, roles, backend APIs, Docker, and MySQL.
- **Hospital Management Modules:** Pharmacy, finance, prescriptions, medicine stock, billing, and payment modules.
- **Road Rescue:** Accident reporting app with emergency status, first-aid guide, medical ID, and notifications.
- **Plot Booking App:** Plot listing, site visit booking, payment, admin approval, and document visibility after approval.
- **Game Projects:** Temple Run-style game experiments, Unity/FBX assets, 3D environment, obstacles, and APK build work.

Personal & Project Overview

1. Tell me about yourself

I am a full-stack and mobile app developer. I mainly work with React Native and Expo for mobile/frontend, Spring Boot or Node.js for backend, and MySQL for database. I have built real-time applications like All Community Matrimony, TraceMyBus, Nari Samman e-commerce, Gated Community app, All Services app, Hospital Management modules, Road Rescue accident alert app, Plot Booking app, and game projects using Unity/React Native.

I have experience in OTP login, JWT authentication, role-based access, admin dashboards, payment gateway integration, Docker deployment, AWS EC2 hosting, MySQL database design, API integration, and APK builds.

2. Explain your main project architecture

Most of my projects follow a 3-layer architecture: frontend, backend, and database.

- Frontend: React Native / Expo or React JS
- Backend: Spring Boot or Node.js Express
- Database: MySQL
- Deployment: Docker Compose on AWS EC2 or Render
- Integrations: Twilio OTP, Razorpay payments, file upload, and notifications

For example, in All Community Matrimony, users register as bride/groom, vendor, or admin. The frontend calls backend REST APIs. The backend validates data, stores it in MySQL, manages authentication using JWT, and returns data to the frontend.

3. What projects have you worked on?

I have worked on multiple real-time projects:

- All Community Matrimony - profiles, vendors, admin approval, payments, subscriptions, language support.
- TraceMyBus - bus tracking details, driver/parent/admin roles, pickup/drop status, SOS, notifications.
- Nari Samman - e-commerce products, cart, COD, admin product management.
- Gated Community App - resident management, OTP login, dashboard, backend deployment.
- All Services / Centra / Servixo - home services booking, providers, payments, admin flow.
- Hospital Management Modules - pharmacy, finance, inventory, billing, stock reduction.
- Road Rescue - accident reporting, emergency status, first-aid guide, medical ID.
- Plot Booking App - plot selection, site visit, payment, admin approval, document visibility.

React Native / Expo / Frontend

4. What is React Native?

React Native is a framework used to build mobile applications using JavaScript and React. It allows us to create Android and iOS apps from a single codebase. In my projects, I used React Native with Expo to build screens, navigation, forms, dashboards, and API-connected mobile apps.

5. Why did you use Expo?

I used Expo because it makes React Native development faster. It provides simple commands to run apps on Android, web, and mobile devices. It also supports EAS Build for APK generation. Expo helped me quickly test UI screens, connect backend APIs, and build APK files.

6. Difference between React Native and React JS

React JS is mainly used for web applications and uses HTML elements like div, input, and button.

React Native is used for mobile applications and uses components like View, Text, TextInput, ScrollView, and TouchableOpacity. In my projects, I used React Native for mobile apps and sometimes Expo web to run the same app in a browser.

7. What is navigation in React Native?

Navigation is used to move from one screen to another. I used React Navigation with stack navigation and bottom tabs.

For example, in my matrimony app, after login the user moves to dashboard. From dashboard, the user can open profile, services, notifications, payment, or settings screens.

```
navigation.navigate("Profile");  
navigation.replace("MainTabs");
```

8. What is useState?

useState is a React hook used to store and update local component data. I used it in forms like registration, login, profile update, product add form, and service booking screens.

```
const [mobile, setMobile] = useState("");
```

9. What is useEffect?

useEffect is used to run code when a component loads or when some data changes. I used useEffect to call backend APIs when a screen opens.

```
useEffect(() => {  
  fetchProducts();  
}, []);
```

10. How do you call backend API from React Native?

I use fetch or axios to call backend APIs.

```
const response = await fetch(`${API_BASE_URL}/api/products`);  
const data = await response.json();
```

In my apps, I stored backend URL in a config file or environment variable.

```
EXPO_PUBLIC_API_BASE_URL=http://server-ip:8080
```

11. Why should we not use localhost in mobile app backend URL?

In mobile apps, localhost means the mobile device itself, not my computer or server. So the app cannot reach the backend.

For real-device testing, I use my system IP or public server IP.

```
http://192.168.1.5:8080  
http://ec2-public-ip:8080
```

Backend / REST API / Security

12. What is Spring Boot?

Spring Boot is a Java framework used to create backend applications quickly. I used Spring Boot to build REST APIs, connect MySQL database, handle authentication, business logic, payments, and admin/customer/vendor operations.

13. What is REST API?

REST API is a way for frontend and backend to communicate using HTTP methods.

```
GET    - fetch data  
POST   - create data  
PUT    - update data  
DELETE - delete data  
  
POST /api/auth/login  
GET  /api/products  
POST /api/bookings  
PUT  /api/admin/approve-user
```

14. Explain JWT authentication

JWT means JSON Web Token. After login, backend generates a token and sends it to frontend. The frontend stores the token and sends it in every protected API request.

```
Authorization: Bearer <token>
```

Backend validates the token and identifies the logged-in user. I used JWT in role-based apps like matrimony, gated community, school, and services apps.

15. What is role-based access?

Role-based access means different users can access different features based on their role.

- Admin: approve users, view revenue, manage vendors.
- Customer: register, book service, make payment.
- Vendor: manage services, accept or reject request.
- Driver: update bus location, pickup/drop status.
- Parent: view bus location details.

Backend checks the user role before allowing access.

Database / MySQL / JPA

16. How did you design database tables?

I designed database tables based on project features.

Matrimony example:

users
profiles
vendors
services
bookings
payments
subscriptions
notifications

TraceMyBus example:

users
schools
drivers
buses
routes
students
location_updates
notifications
sos_alerts

Each table has primary keys and relationships using foreign keys.

17. What is MySQL?

MySQL is a relational database. I used MySQL to store user data, profiles, bookings, products, payments, admin approvals, and service records. I used MySQL because it is reliable, structured, and good for real-time business applications.

18. What is a foreign key?

A foreign key connects one table with another table.

```
bookings.user_id -> users.id  
payments.booking_id -> bookings.id
```

This helps maintain relation between user, booking, and payment.

19. What is Hibernate / JPA?

Hibernate/JPA is used in Spring Boot to map Java classes to database tables.

```
@Entity  
public class User {  
    @Id  
    @GeneratedValue  
    private Long id;  
}
```

Instead of writing raw SQL every time, we can use repository methods.

```
userRepository.findByMobile(mobile);
```

20. What is ddl-auto=update?

In Spring Boot, this setting updates database tables automatically based on entity classes.

```
spring.jpa.hibernate.ddl-auto=update
```

I used it during development. But in production, it is better to use migrations like Flyway or manually controlled schema updates.

Errors / CORS / OTP / Payment

21. What is CORS?

CORS means Cross-Origin Resource Sharing. It allows frontend running on one port or domain to call backend running on another port or domain.

```
Frontend: http://localhost:8081
Backend: http://localhost:8080
```

If CORS is not configured, the browser blocks the API request. In Spring Boot, I configured allowed origins for frontend URL.

22. Why do we get 500 Internal Server Error?

500 error means backend code failed. Common reasons are:

- Database connection failed.
- Null pointer exception.
- Wrong API request body.
- Missing environment variable.
- Payment or OTP key issue.
- SQL error.
- Backend service dependency issue.

I debug it by checking backend logs, API payload, database connection, and exception message.

23. What is the difference between 400, 401, 403, 404, and 500?

```
400 - Bad request, wrong input
401 - Unauthorized, login/token missing
403 - Forbidden, user has no permission
404 - API not found
500 - Backend server error
```

These errors helped me debug login, admin access, OTP, and payment APIs.

24. Explain OTP login flow

OTP login flow:

1. User enters mobile number.
2. Frontend calls send OTP API.
3. Backend sends OTP using Twilio Verify.
4. User enters OTP.
5. Backend verifies OTP.
6. If OTP is correct, backend generates JWT token.
7. User enters dashboard.

In development, dummy OTP can be used. But in production, random OTP should not be accepted. It must be verified through Twilio.

25. What is Twilio Verify?

Twilio Verify is a service used to send and verify OTP by SMS. It provides a Verify Service SID. Backend uses Twilio account SID, auth token, and verify SID to send OTP and validate OTP. I used it for real mobile number verification in login and registration flows.

26. What is Razorpay integration?

Razorpay is a payment gateway. I used it for online payments in projects like matrimony, hospital pharmacy, services app, and plot booking.

Basic flow:

1. Backend creates payment order.
2. Frontend opens Razorpay checkout.
3. User pays.
4. Razorpay returns payment ID, order ID, and signature.
5. Backend verifies signature.
6. Payment status is saved in MySQL.

27. Why should payment verification happen in backend?

Payment verification must happen in backend because frontend can be modified by users. Backend safely verifies Razorpay signature using the secret key. This prevents fake payment success.

Deployment / Docker / AWS

28. What is Docker?

Docker is used to run applications inside containers. I used Docker to run frontend, backend, and MySQL together. This makes deployment easier and avoids works-on-my-system issues.

29. What is Docker Compose?

Docker Compose is used to manage multiple containers using one docker-compose.yml file.

```
services:
  mysql:
  backend:
  frontend:
```

I used Docker Compose for projects like gated community, all services, matrimony deployment, and Nari Samman.

30. What is AWS EC2?

AWS EC2 is a cloud server. I used EC2 to deploy backend, frontend, and MySQL using Docker. After deployment, the frontend is opened using public IP and the backend APIs are accessed through public backend URL.

31. How do you deploy your full-stack app?

My deployment flow:

1. Push code to GitHub.
2. Connect to EC2 server.
3. Install Docker and Docker Compose.
4. Add backend .env and frontend .env.
5. Run docker-compose build and start containers.
6. Check containers with docker ps.
7. Test backend with curl.
8. Open frontend in browser.

```
docker-compose up -d --build
docker ps
curl http://localhost:8080/api/test
```

32. What deployment problems have you solved?

I solved many real-time deployment issues:

- Port conflicts like 8080, 8081, and 8082.
- CORS blocked API calls.
- Frontend using localhost instead of server IP.
- MySQL container not healthy.
- Backend not connecting to database.
- Missing environment variables.
- Expo web MIME type error.
- Docker container restart issues.
- Spring Boot compilation errors.
- APK backend connection issue.

I debugged them using logs, curl, docker ps, docker logs, and environment variable checks.

33. How do you check backend logs in Docker?

```
docker logs container_name  
docker logs -f container_name
```

Example:

```
docker logs qlearo_backend  
docker logs -f qlearo_backend
```

34. How do you check if MySQL is running?

On Windows:

```
Get-Service *mysql*
```

In Docker:

```
docker ps  
docker logs mysql_container
```

Inside MySQL:

```
mysql -u root -p  
SHOW DATABASES;
```

35. What is environment variable?

Environment variables store configuration values outside the code. This is safer than hardcoding secrets inside source code.

```
MYSQL_DATABASE=qlearo_db  
MYSQL_USER=qlearo_user  
MYSQL_PASSWORD=*****  
JWT_SECRET=*****  
RAZORPAY_KEY_ID=*****  
TWILIO_ACCOUNT_SID=*****
```

Business Flows / Notifications / Language

36. What is admin approval flow?

Admin approval flow means user data is not visible or active until admin approves it.

Example from plot booking:

1. Customer selects plot.
2. Customer makes payment.
3. Admin verifies payment/documents.
4. Admin approves.

5. Customer can view registration documents.

This protects business process and avoids unauthorized access.

37. How did you handle document visibility?

I used status-based access.

```
PENDING
PAID
UNDER_REVIEW
APPROVED
REJECTED
```

Only when status is APPROVED, frontend shows registration documents. Backend also checks the status before returning document data.

38. What is notification system?

Notification system informs users about important actions.

- Admin approved profile.
- Vendor accepted booking.
- Payment successful.
- Bus delayed.
- SOS alert raised.
- Document approved.

In my projects, notifications are stored in MySQL and displayed in notification screens. Push notifications can be added later using FCM.

39. How did you handle multilingual support?

I worked on English to Telugu language support. For static app text, translation keys can be used. For dynamic admin-added data, backend translation can be used and both English and Telugu values can be saved in MySQL.

```
title_en
title_te
description_en
description_te
```

This avoids translating the same text every time.

40. What is the best way to translate newly added data?

For newly added data, backend should translate at the time of saving and store both languages in database.

```
name_en = Honey
name_te = હોની
```

Then frontend displays based on selected language.

APK / Project Explanations / Debugging

41. What is Expo EAS Build?

EAS Build is Expo's cloud build service used to create APK or AAB files.

```
eas build --platform android --profile preview
```

I used it to generate APK files for mobile app testing.

42. What is the difference between APK and AAB?

APK is used for direct installation and testing on Android phones. AAB is Android App Bundle used for Play Store upload. For testing, I use APK. For Play Store release, I use AAB.

43. What is your strongest project?

My strongest project is All Community Matrimony because it includes real business features like:

- Customer registration.
- Bride/groom profiles.
- Vendor services.
- Admin dashboard.
- OTP login.
- Payment flow.
- Subscriptions.
- Language support.
- Notifications.
- MySQL backend.
- Deployment.

It shows my full-stack development skills from UI to backend to database and deployment.

44. Explain All Community Matrimony project

All Community Matrimony is a matrimony and wedding services application. Users can register as bride/groom, create profiles, search matches, save profiles, send interest requests, and use wedding services

like hall, catering, decoration, and orchestra. Vendors can manage their services and bookings. Admin can manage users, vendors, subscriptions, payments, and approvals. The app uses React Native frontend, Spring Boot backend, MySQL database, JWT authentication, OTP verification, and payment integration.

45. Explain TraceMyBus project

TraceMyBus is a school bus tracking and safety application. It has roles like admin, driver, parent, school, and employee. Driver can update pickup/drop status and location details. Parents can see live location details, bus status, delay reason, and emergency updates. The app also supports SOS alerts, notifications, and role-based access.

46. Explain Nari Samman project

Nari Samman is an e-commerce application for products like honey, spices, handicrafts, sarees, and tribal products. It includes product listing, product details, cart, cash on delivery, admin product management, and backend database connection. It uses React frontend/mobile UI, Spring Boot backend, and MySQL database.

47. Explain Hospital Management module

In hospital management, I worked on modules like pharmacy and finance. Pharmacy includes medicine inventory, prescription handling, medicine delivery to patient, stock search, stock reduction, and payment. Finance includes bills, revenue stats, monthly trends, income sources, and mark-paid feature.

48. Explain Plot Booking project

Plot Booking app allows customers to view plots, book site visits, select plots, make payment, and view registration documents after admin approval. Admin manages plots, site visits, payments, approvals, and uploaded documents.

49. What is your role in these projects?

I worked on the full development flow:

- Requirement understanding.
- Screen design.
- Frontend implementation.
- Backend API creation.
- Database table design.
- Authentication.
- Payment/OTP integration.
- Testing.

- Debugging.
- Deployment.
- APK build.

I also fixed real-time errors in frontend, backend, database, and deployment.

50. What are common errors you faced in React Native?

I faced errors like:

- JSX closing tag missing.
- ScrollView not closed.
- Navigation screen not found.
- API not loading.
- Network request failed.
- AsyncStorage issue.
- Metro bundler error.
- MIME type error in Expo web.
- Environment URL issue.

I solved them by checking component structure, imports, navigation names, API base URL, and Metro cache.

51. How do you fix JSX closing tag error?

I check whether every opening tag has a closing tag.

```
<ScrollView>
  <View>
    <Text>Hello</Text>
  </View>
</ScrollView>
```

If ScrollView or View is not closed properly, React Native gives JSX error.

52. What is API base URL issue?

API base URL issue happens when frontend points to the wrong backend URL.

Wrong for mobile/server:

```
http://localhost:8080
```

Correct for mobile/server:

```
http://server-ip:8080
```

I fixed this in many deployed apps by updating EXPO_PUBLIC_API_BASE_URL.

53. How do you secure backend APIs?

I secure backend APIs using:

- JWT token authentication.
- Role-based authorization.
- Password hashing.
- Backend payment verification.
- Input validation.
- CORS control.
- Environment variables for secrets.

54. What is password hashing?

Password hashing means converting password into a secure hash before saving in database. In Spring Boot, BCrypt is commonly used. We should never store plain passwords.

55. How do you test APIs?

I test APIs using:

- Postman.
- curl.
- Browser for GET APIs.
- Frontend form testing.
- Backend logs.
- Database verification.

```
curl http://localhost:8080/api/products
```

56. What is CI/CD?

CI/CD means Continuous Integration and Continuous Deployment. It helps automatically build and deploy code after pushing to GitHub. In my projects, I worked with GitHub Actions and deployment scripts to deploy apps on EC2 using Docker Compose.

57. What is GitHub Actions?

GitHub Actions is used to automate build, test, and deployment. For example, when code is pushed to the main branch, GitHub Actions can connect to the server and run Docker Compose build/deploy commands.

58. What is the difference between frontend and backend?

Frontend is what the user sees and interacts with: screens, forms, buttons, dashboards. Backend handles business logic, database, authentication, payments, OTP, approvals, and APIs.

59. What is your real-time debugging process?

My debugging process is:

1. Check frontend error.
2. Check API URL.
3. Test API separately.
4. Check backend logs.
5. Check database connection.
6. Check request body.
7. Check environment variables.
8. Fix and retest.

This helped me solve many issues like 500 error, CORS error, OTP failure, payment failure, and deployment issues.

60. Why should we hire you?

You should hire me because I have hands-on experience building real-time full-stack applications. I can work on frontend, backend, database, deployment, and debugging. I have built apps with authentication, admin panels, payments, OTP, role-based access, MySQL, Docker, AWS, and APK builds. I am also comfortable solving real-time errors and completing end-to-end project flow from development to deployment.

Best 2-Minute Project Explanation

I have worked on multiple full-stack projects using React Native/Expo, Spring Boot/Node.js, MySQL, Docker, and AWS. My main project is All Community Matrimony, where users can register, create profiles, search matches, use wedding services, make payments, and get admin approval. I implemented frontend screens, backend REST APIs, MySQL database tables, JWT authentication, OTP login, role-based access, payment integration, and deployment. I also worked on TraceMyBus, Nari Samman e-commerce, Gated Community, Hospital modules, All Services app, and Plot Booking app. Through these projects, I gained real-time experience in API integration, debugging, CORS, Docker deployment, MySQL connection, APK build, and production issue fixing.

Most Important Topics to Revise First

1. React Native components and navigation
2. useState, useEffect, and API calls
3. Spring Boot REST APIs
4. MySQL tables and relationships
5. JWT authentication
6. Role-based access
7. OTP flow with Twilio
8. Razorpay payment flow
9. Docker and Docker Compose
10. AWS EC2 deployment
11. CORS and API URL issues
12. Debugging 400, 401, 403, 404, and 500 errors

Short Closing Answer for HR Round

I am ready to work on real-time full-stack projects. I can understand requirements, build frontend screens, create backend APIs, design MySQL tables, integrate OTP and payments, debug issues, and deploy the application. I have already handled real project problems, so I can contribute from development to production support.